1. To Apply Escape Sequences

Store Command History

Navigate Previous Commands

Navigate Next Commands

Update Input Buffer

Test Recall Functionality

2. To Allocate Buffers Dynamically

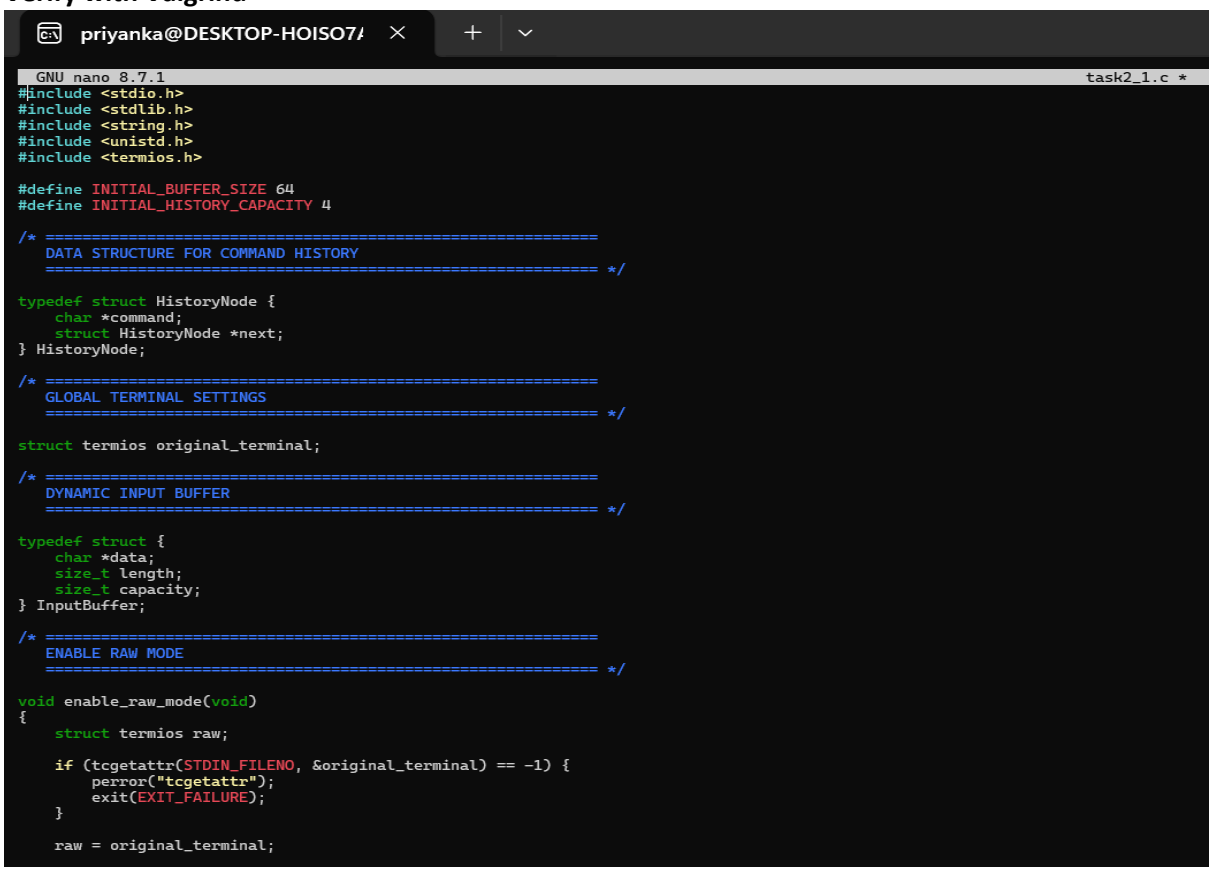
Resize Arrays

Prevent Buffer Overflow

Manage Linked Lists

Release Memory Correctly

Verify with Valgrind



priyanka@DESKTOP-HOISO7A

GNU nano 8.7.1

```
    raw.c_lflag &= ~(ICANON | ECHO);

    /*
     * Read one character at a time.
     */
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    if (tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw) == -1) {
        perror("tcsetattr");
        exit(EXIT_FAILURE);
    }
}

/* ============================================================
   RESTORE TERMINAL
   ============================================================ */

void disable_raw_mode(void)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_terminal);
}

/* ============================================================
   INITIALIZE INPUT BUFFER
   ============================================================ */

void init_buffer(InputBuffer *buffer)
{
    buffer->capacity = INITIAL_BUFFER_SIZE;
    buffer->length = 0;

    buffer->data = malloc(buffer->capacity);

    if (buffer->data == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    buffer->data[0] = '\0';
}

/* ============================================================
   RESIZE INPUT BUFFER
   ============================================================ */

void resize_buffer(InputBuffer *buffer)
{
    size_t new_capacity = buffer->capacity * 2;

    char *new_data = realloc(buffer->data, new_capacity);

    if (new_data == NULL) {
        perror("realloc");
```

```
 GNU nano 8.7.1
        free(buffer->data);
        exit(EXIT_FAILURE);
    }

    buffer->data = new_data;
    buffer->capacity = new_capacity;
}

/* ============================================================
   CLEAR INPUT BUFFER
   ============================================================ */

void clear_buffer(InputBuffer *buffer)
{
    buffer->length = 0;

    if (buffer->data != NULL) {
        buffer->data[0] = '\0';
    }
}

/* ============================================================
   SET INPUT BUFFER
   Used when recalling history.
   ============================================================ */

void set_buffer(InputBuffer *buffer, const char *text)
{
    size_t required = strlen(text) + 1;

    while (buffer->capacity < required) {
        resize_buffer(buffer);
    }

    strcpy(buffer->data, text);
    buffer->length = strlen(text);
}

/* ============================================================
   APPEND CHARACTER TO INPUT BUFFER
   ============================================================ */

void append_character(InputBuffer *buffer, char c)
{
    /*
     * Leave one byte for '\0'.
     */
    if (buffer->length + 1 >= buffer->capacity) {
        resize_buffer(buffer);
    }

    buffer->data[buffer->length] = c;
    buffer->length++;
```

```
  GNU nano 8.7.1
    buffer->data[buffer->length] = '\0';
}

/* ============================================================
   REMOVE LAST CHARACTER
   ============================================================ */

void remove_last_character(InputBuffer *buffer)
{
    if (buffer->length > 0) {
        buffer->length--;
        buffer->data[buffer->length] = '\0';

        /*
         * Erase character visually from terminal.
         */
        printf("\b \b");
        fflush(stdout);
    }
}

/* ============================================================
   FREE INPUT BUFFER
   ============================================================ */

void free_buffer(InputBuffer *buffer)
{
    free(buffer->data);

    buffer->data = NULL;
    buffer->length = 0;
    buffer->capacity = 0;
}

/* ============================================================
   ADD COMMAND TO LINKED LIST HISTORY
   ============================================================ */

void add_history(HistoryNode **head, const char *command)
{
    HistoryNode *new_node;

    /*
     * Don't store empty commands.
     */
    if (command[0] == '\0') {
        return;
    }

    new_node = malloc(sizeof(HistoryNode));

    if (new_node == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
```

priyanka@DESKTOP-HOISO7A

GNU nano 8.7.1

```
    }

    strcpy(new_node->command, command);

    new_node->next = NULL;

    /*
     * Add at end of linked list.
     */
    if (*head == NULL) {
        *head = new_node;
    }
    else {
        HistoryNode *current = *head;

        while (current->next != NULL) {
            current = current->next;
        }

        current->next = new_node;
    }
}

/* ============================================================
   COUNT HISTORY ITEMS
   ============================================================ */

int history_count(HistoryNode *head)
{
    int count = 0;

    while (head != NULL) {
        count++;
        head = head->next;
    }

    return count;
}

/* ============================================================
   GET COMMAND AT INDEX
   ============================================================ */

const char *get_history_command(HistoryNode *head, int index)
{
    int current_index = 0;

    while (head != NULL) {
        if (current_index == index) {
            return head->command;
        }

        current_index++;
        head = head->next;
```

priyanka@DESKTOP-HOISO7A

```
  GNU nano 8.7.1
    return NULL;
}

/* ============================================================
   DISPLAY HISTORY
   ============================================================ */

void display_history(HistoryNode *head)
{
    int number = 1;

    if (head == NULL) {
        printf("\nNo command history.\n");
        return;
    }

    printf("\nCommand History:\n");

    while (head != NULL) {
        printf("%d  %s\n", number, head->command);

        number++;
        head = head->next;
    }
}

/* ============================================================
   FREE LINKED LIST HISTORY
   ============================================================ */

void free_history(HistoryNode *head)
{
    HistoryNode *current = head;

    while (current != NULL) {
        HistoryNode *next = current->next;

        free(current->command);
        free(current);

        current = next;
    }
}

/* ============================================================
   CLEAR CURRENT TERMINAL LINE
   ============================================================ */

void clear_current_line(InputBuffer *buffer)
{
    /*
     * Move cursor to beginning.
     */
    printf("\r");
```

```
 GNU nano 8.7.1
    printf("myshell> ");

    for (size_t i = 0; i < buffer->length; i++) {
        printf(" ");
    }

    /*
     * Return to beginning again.
     */
    printf("\rmyshell> ");

    fflush(stdout);
}

/* ============================================================
   REDRAW INPUT
   ============================================================ */

void redraw_input(InputBuffer *buffer)
{
    printf("\r");

    /*
     * Clear old line.
     */
    printf("myshell> ");

    for (size_t i = 0; i < buffer->length + 20; i++) {
        printf(" ");
    }

    printf("\r");
    printf("myshell> ");
    printf("%s", buffer->data);

    fflush(stdout);
}

/* ============================================================
   READ USER INPUT
   ============================================================ */

int read_input(
    InputBuffer *buffer,
    HistoryNode *history,
    char **saved_current_input
)
{
    char c;
```

```
GNU nano 8.7.1
    int history_index = -1;

    int total_history = history_count(history);

    while (1) {

        if (read(STDIN_FILENO, &c, 1) != 1) {
            return -1;
        }

        /* ======================================================
           ENTER KEY
           ====================================================== */

        if (c == '\n' || c == '\r') {

            printf("\n");

            return 0;
        }

        /* ======================================================
           BACKSPACE
           ====================================================== */

        if (c == 127 || c == 8) {

            /*
             * If user is navigating history, editing starts
             * a new input state.
             */
            history_index = -1;

            if (buffer->length > 0) {
                remove_last_character(buffer);
            }

            continue;
        }

        /* ======================================================
           CTRL + D
           ====================================================== */

        if (c == 4) {
            printf("\n");
            return 1;
        }
```

```c
if (c == 27) {

    char sequence[2];

    if (read(STDIN_FILENO, &sequence[0], 1) != 1) {
        continue;
    }

    if (read(STDIN_FILENO, &sequence[1], 1) != 1) {
        continue;
    }

    /* ================================================
       ARROW UP
       ================================================ */

    if (sequence[0] == '[' && sequence[1] == 'A') {

        if (total_history == 0) {
            continue;
        }

        /*
         * Save current unfinished input before
         * entering history navigation.
         */
        if (history_index == -1) {

            free(*saved_current_input);

            *saved_current_input =
                malloc(buffer->length + 1);

            if (*saved_current_input == NULL) {
                perror("malloc");
                exit(EXIT_FAILURE);
            }

            strcpy(*saved_current_input, buffer->data);

            history_index = total_history - 1;
        }
        else if (history_index > 0) {
            history_index--;
        }

        {
            const char *command =
                get_history_command(
                    history,
                    history_index
                );
```

priyanka@DESKTOP-HOISO7/

GNU nano 8.7.1

```
                if (command != NULL) {
                    clear_current_line(buffer);
                    set_buffer(buffer, command);
                    redraw_input(buffer);
                }
            }

            continue;
        }

        /* ================================================
           ARROW DOWN
           ================================================ */

        if (sequence[0] == '[' && sequence[1] == 'B') {

            if (history_index == -1) {
                continue;
            }

            if (history_index < total_history - 1) {

                history_index++;

                {
                    const char *command =
                        get_history_command(
                            history,
                            history_index
                        );

                    if (command != NULL) {
                        clear_current_line(buffer);
                        set_buffer(buffer, command);
                        redraw_input(buffer);
                    }
                }
            }
            else {

                /*
                 * Reached the newest command.
                 * Restore the unfinished command.
                 */
                history_index = -1;

                clear_current_line(buffer);

                if (*saved_current_input != NULL) {
                    set_buffer(
                        buffer,
                        *saved_current_input
                    );
```

priyanka@DESKTOP-HOISO7A

GNU nano 8.7.1

```
                    }
                    else {
                        clear_buffer(buffer);
                    }

                    redraw_input(buffer);
                }

                continue;
            }

            continue;
        }

        /* ================================================
           NORMAL PRINTABLE CHARACTER
           ================================================ */

        if (c >= 32 && c <= 126) {

            /*
             * Once user types something, history navigation
             * is reset.
             */
            history_index = -1;

            append_character(buffer, c);

            putchar(c);
            fflush(stdout);
        }
    }
}

/* ============================================================
   PROCESS COMMAND
   ============================================================ */

int process_command(
    InputBuffer *buffer,
    HistoryNode **history
)
{
    if (buffer->length == 0) {
        return 0;
    }

    /*
     * exit command
     */
    if (strcmp(buffer->data, "exit") == 0) {
        return 1;
    }
```

```
 GNU nano 8.7.1
        if (result == 1 || result == -1) {
            break;
        }

        /*
         * Store non-empty commands in history.
         */
        if (input.length > 0) {
            add_history(
                &history,
                input.data
            );
        }

        /*
         * Process command.
         */
        exit_requested =
            process_command(
                &input,
                &history
            );

        /*
         * Reset input buffer for next command.
         */
        clear_buffer(&input);

        /*
         * Reset saved unfinished command.
         */
        free(saved_current_input);
        saved_current_input = NULL;
    }

    /*
     * Restore terminal before printing final message.
     */
    disable_raw_mode();

    printf("Exiting shell...\n");

    /*
     * Release all dynamically allocated memory.
     */
    free_buffer(&input);

    free(saved_current_input);

    free_history(history);

    return 0;
}
```

Output:

```
priyanka@DESKTOP-HOISO7A:~$ cd ~/process_lab
nano task2_1.c
priyanka@DESKTOP-HOISO7A:~/process_lab$ gcc -Wall -Wextra -std=c11 task2_1.c -o task2_1
priyanka@DESKTOP-HOISO7A:~/process_lab$ ./task2_1
myshell> hello
Hello! Welcome to the Week 2 shell.
myshell> pid
Process ID: 1893
myshell> help

Available commands:
  help     - Display help
  history  - Display command history
  hello    - Display greeting
  pid      - Display process ID
  exit     - Exit program

myshell> history

Command History:
1  hello
2  pid
3  help
4  history
```